

Ejercicios del grupo 3

1.- Suma de elementos en vector: Leer un vector de enteros y calcular la suma total.

Algoritmo Suma_Vector

Definir vector como lista de enteros

Definir suma como entero inicializado en 0

Escribir "Ingrese la cantidad de elementos del vector:"

Leer n

Para i desde 1 hasta n hacer

 Escribir "Ingrese el elemento ", i

 Leer vector[i]

FinPara

Para i desde 1 hasta n hacer

 suma $\leftarrow$ suma + vector[i]

FinPara

Escribir "La suma total de los elementos es: ", suma

FinAlgoritmo

PRUEBA DE ESCRITORIO: Vector = [4, 7, 2, 9, 3]

Iteración	Elemento leído	Suma acumulada
Inicio	-	0
i = 1	4	0 + 4 = 4
i = 2	7	4 + 7 = 11
i = 3	2	11 + 2 = 13
i = 4	9	13 + 9 = 22
i = 5	3	22 + 3 = 25

La suma total de los elementos es: 25

2.- Promedio de notas: Ingresar notas en un vector y calcular el promedio.

Algoritmo Promedio_Notas

Definir vector como lista de reales

Definir suma como real inicializado en 0

Definir promedio como real

Escribir "Ingrese la cantidad de notas:"

Leer n

Para i desde 1 hasta n hacer

```

    Escribir "Ingrese la nota ", i
    Leer vector[i]
    suma ← suma + vector[i]
FinPara

promedio ← suma / n

Escribir "El promedio de las notas es: ", promedio
FinAlgoritmo

```

PRUEBA DE ESCRITORIO: Vector Notas = [16, 20, 11, 08] (En este caso se trabajará con 4 notas)

Iteración	Nota ingresada	Suma acumulada
Inicio	-	0
i = 1	16	0 + 16 = 16
i = 2	20	16 + 20 = 36
i = 3	11	36 + 11 = 47
i = 4	08	47 + 08 = 55

Cálculo de promedio: $55/4 = 13.75$
 El promedio de las notas es: 13.75

3.- Matriz identidad: Generar una matriz identidad de tamaño n.

```

Algoritmo Matriz_Identidad
  Definir matriz como arreglo bidimensional de tamaño n x n
  Escribir "Ingrese el tamaño de la matriz identidad:"
  Leer n

  Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
      Si i = j Entonces
        matriz[i][j] ← 1
      Sino
        matriz[i][j] ← 0
      FinSi
    FinPara
  FinPara

  Escribir "La matriz identidad de tamaño ", n, " es:"
  Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
      Escribir matriz[i][j] con espacio
    FinPara
    Saltar línea
  FinPara
FinAlgoritmo

```

PRUEBA DE ESCRITORIO: n = 4

Matriz identidad de n x n =

i \ j	1	2	3	4
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

4.- Máximo y mínimo: Encontrar el mayor y menor valor en un vector.

Algoritmo Maximo_Minimo

Definir vector como lista de enteros

Definir mayor y menor como enteros

Escribir "Ingrese la cantidad de elementos del vector:"

Leer n

Para i desde 1 hasta n hacer

 Escribir "Ingrese el elemento ", i

 Leer vector[i]

FinPara

mayor $\leftarrow$ vector[1]

menor $\leftarrow$ vector[1]

Para i desde 2 hasta n hacer

 Si vector[i] > mayor Entonces

 mayor $\leftarrow$ vector[i]

 FinSi

 Si vector[i] < menor Entonces

 menor $\leftarrow$ vector[i]

 FinSi

FinPara

Escribir "El valor máximo es: ", mayor

Escribir "El valor mínimo es: ", menor

FinAlgoritmo

PRUEBA DE ESCRITORIO: Vector = [9, 1, 5, 17, 20]

Iteración	Elemento	Mayor actual	Menor actual
Inicio	-	0	0
i = 1	9	9	9
i=2	1	9	1
i=3	5	9	1
i=4	17	17	1
i=5	20	20	1

5.- Función factorial: Implementar una función que calcule el factorial de un número.

Algoritmo Factorial

Definir n como entero

Definir f como entero inicializado en 1

Escribir "Ingrese un número entero positivo:"

Leer n

Para i desde 1 hasta n hacer

$f \leftarrow f * i$

FinPara

Escribir "El factorial de ", n, " es: ", f

FinAlgoritmo

PRUEBA DE ESCRITORIO: n = 5

Iteración	i	(f) acumulado
Inicio	-	1
i = 1	1	1 * 1 = 1
i = 2	2	1 * 2 = 2
i = 3	3	2 * 3 = 6
i = 4	4	6 * 4 = 24
i = 5	5	24 * 5 = 120

El factorial de 5 es: 120

6.- Ordenamiento burbuja: Implementar Bubble Sort en una cadena de caracteres.

Algoritmo Burbuja_Cadena

Definir cadena como texto

Definir arreglo de caracteres

Escribir "Ingrese una cadena de caracteres:"

Leer cadena

Convertir cadena en arreglo de caracteres

Para i desde 1 hasta longitud(cadena)-1 hacer

Para j desde 1 hasta longitud(cadena)-i hacer

Si arreglo[j] > arreglo[j+1] Entonces

Intercambiar arreglo[j] con arreglo[j+1]

FinSi

FinPara

FinPara

Escribir "La cadena ordenada es: ", arreglo convertido a texto

FinAlgoritmo

PRUEBA DE ESCRITORIO: Se ingresa el mensaje "true"

Iteración externa (i)	Comparaciones	Condición	Resultado parcial
i = 1	t vs r $\rightarrow$ t > r	Intercambia	rtue
	t vs u $\rightarrow$ t < u	No cambia	rtue
	u vs e $\rightarrow$ u > e	Intercambia	rteu
i = 2	r vs t $\rightarrow$ r < t	No cambia	rteu
	t vs e $\rightarrow$ t > e	Intercambia	retu
	t vs u $\rightarrow$ t < u	No cambia	retu
i = 3	r vs e $\rightarrow$ r > e	Intercambia	ertu
	r vs t $\rightarrow$ r < t	No cambia	ertu
	t vs u $\rightarrow$ t < u	No cambia	ertu

7.- Ordenamiento por inserción: Implementar Insertion Sort en un vector de enteros.

Algoritmo Insercion_Vector

Definir vector como lista de enteros

Escribir "Ingrese la cantidad de elementos:"

Leer n

Para i desde 1 hasta n hacer
 Escribir "Ingrese el elemento ", i
 Leer vector[i]
FinPara

Para i desde 2 hasta n hacer
 clave ← vector[i]
 j ← i - 1

 Mientras j ≥ 1 y vector[j] > clave hacer
 vector[j+1] ← vector[j]
 j ← j - 1
 FinMientras

 vector[j+1] ← clave
FinPara

 Escribir "Vector ordenado: ", vector
FinAlgoritmo

PRUEBA DE ESCRITORIO: Vector = [7, 4, 6, 1]

Iteración	Elemento (clave)	Comparación	Resultado parcial
i = 1	7	No se hace comparación al ser el primer elemento.	[7, 4, 6, 1]
i = 2	4	7 > 4 se mueve 7	[4, 7, 6, 1]
i = 3	6	7 > 6 se mueve 7	[4, 6, 7, 1]
i = 4	1	7 > 1 se mueve 7	[4, 6, 1, 7]
		6 > 1 se mueve 6	[4, 1, 6, 7]
		4 > 1 se mueve 4	[1, 4, 6, 7]

8.- Transposición de matriz: Calcular la matriz transpuesta de una matriz n x m.

Algoritmo Transposicion_Matriz
 Definir matriz[n][m]
 Definir transpuesta[m][n]

 Escribir "Ingrese número de filas (n):"
 Leer n
 Escribir "Ingrese número de columnas (m):"
 Leer m

 Para i desde 1 hasta n hacer
 Para j desde 1 hasta m hacer
 Escribir "Elemento [i][j]:"
 Leer matriz[i][j]
 FinPara
 FinPara

 Para i desde 1 hasta n hacer

```

    Para j desde 1 hasta m hacer
        transpuesta[j][i] ← matriz[i][j]
    FinPara
FinPara

```

```

Escribir "Matriz transpuesta:"
Mostrar transpuesta
FinAlgoritmo

```

PRUEBA DE ESCRITORIO: Matriz de tamaño 3 * 2 M = $\begin{bmatrix} 7 & 8 \\ 1 & 2 \\ 4 & 5 \end{bmatrix}$

Paso de transposición:

- $M[1][1]=7 \rightarrow T[1][1]$
- $M[1][2]=8 \rightarrow T[2][1]$
- $M[2][1]=1 \rightarrow T[1][2]$
- $M[2][2]=2 \rightarrow T[2][2]$
- $M[3][1]=4 \rightarrow T[1][3]$
- $M[3][2]=5 \rightarrow T[2][3]$

RESULTADO: T = $\begin{bmatrix} 7 & 1 & 4 \\ 8 & 2 & 5 \end{bmatrix}$

9.- Multiplicación de matrices: Multiplicar dos matrices cuadradas.

Algoritmo Multiplicacion_Matrices
 Definir A[n][n], B[n][n], C[n][n]

```

Escribir "Ingrese el tamaño de la matriz (n):"
Leer n

```

```

Escribir "Ingrese los elementos de la matriz A:"
Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
        Leer A[i][j]
    FinPara
FinPara

```

```

Escribir "Ingrese los elementos de la matriz B:"
Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
        Leer B[i][j]
    FinPara
FinPara

```

```

Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
        C[i][j] ← 0
        Para k desde 1 hasta n hacer
            C[i][j] ← C[i][j] + A[i][k] * B[k][j]
        FinPara
    FinPara
FinPara

```

```

Escribir "La matriz resultante C es:"

```

Mostrar C
FinAlgoritmo

PRUEBA DE ESCRITORIO: $n = 2$ $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

Multiplicar:

1. $C[1][1] = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19$
2. $C[1][2] = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22$
3. $C[2][1] = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43$
4. $C[2][2] = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50$

Resultado: $C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

10.- Vector dinámico con apuntadores: Crear un vector dinámico usando new y delete.

Algoritmo Vector_Dinamico

Definir puntero vector

Escribir "Ingrese tamaño del vector:"

Leer n

$\text{vector} \leftarrow \text{new entero}[n]$ // reserva dinámica de memoria

Para i desde 1 hasta n hacer

 Escribir "Ingrese elemento ", i

 Leer vector[i]

FinPara

Escribir "Contenido del vector:"

Para i desde 1 hasta n hacer

 Escribir vector[i]

FinPara

delete[] vector // liberar memoria

FinAlgoritmo

11.- Funciones con parámetros por referencia: Implementar una función que intercambie dos valores usando referencias.

Algoritmo Intercambio_Referencia

Definir función Intercambiar(a, b)

 temp $\leftarrow$ a

 a $\leftarrow$ b

 b $\leftarrow$ temp

FinFunción

Escribir "Ingrese primer valor:"

Leer x

Escribir "Ingrese segundo valor:"

Leer y

Lamar Intercambiar(x, y)

```
    Escribir "Después del intercambio:"  
    Escribir "x = ", x, " y = ", y  
FinAlgoritmo
```

PRUEBA DE ESCRITORIO: X = 15; Y = 30  X = 30; Y = 15

12.- Archivo de estudiantes: Leer nombres y notas desde un archivo y calcular el promedio general.

```
Algoritmo Promedio_Archivo  
    Abrir archivo "estudiantes.txt" en modo lectura  
    suma ← 0  
    contador ← 0  
  
    Mientras no sea fin de archivo hacer  
        Leer nombre, nota  
        suma ← suma + nota  
        contador ← contador + 1  
    FinMientras  
  
    promedio ← suma / contador  
    Escribir "Promedio general = ", promedio  
  
    Cerrar archivo  
FinAlgoritmo
```

13.- Archivo de productos: Leer productos y precios desde un archivo y aplicar un descuento.

```
Algoritmo Productos_Descuento  
    Abrir archivo "productos.txt" en modo lectura  
    Mientras no sea fin de archivo hacer  
        Leer nombre, precio  
        Si precio > 999 entonces  
            precioFinal ← precio * 0.8  
            Escribir nombre, precio, precioFinal  
        Sino  
            Escribir nombre, precio, "Sin descuento"  
        FinSi  
    FinMientras  
    Cerrar archivo  
FinAlgoritmo
```

14.- Ordenamiento con funciones: Crear una función que ordene un vector y otra que lo imprima. (Bubble sort)

```
Algoritmo Ordenar_Vector  
    Definir función Ordenar(vector, n)  
        Para i desde 1 hasta n-1 hacer  
            Para j desde 1 hasta n-i hacer  
                Si vector[j] > vector[j+1] entonces  
                    Intercambiar(vector[j], vector[j+1])  
                FinSi  
            FinPara  
        FinPara  
    FinFunción
```



```

Definir función Imprimir(vector, n)
  Para i desde 1 hasta n hacer
    Escribir vector[i]
  FinPara
FinFunción

Leer n
Leer vector[n]

Llamar Ordenar(vector, n)
Llamar Imprimir(vector, n)
FinAlgoritmo

```

15.- Matrices simétricas: Verificar si una matriz cuadrada es simétrica.

```

Algoritmo Matriz_Simetrica
  Leer n
  Definir matriz[n][n]

  Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
      Leer matriz[i][j]
    FinPara
  FinPara

  simetrica ← verdadero
  Para i desde 1 hasta n hacer
    Para j desde 1 hasta n hacer
      Si matriz[i][j] ≠ matriz[j][i] entonces
        simetrica ← falso
      FinSi
    FinPara
  FinPara

  Si simetrica entonces
    Escribir "La matriz es simétrica"
  Sino
    Escribir "La matriz NO es simétrica"
  FinSi
FinAlgoritmo

```

PRUEBA DE ESCRITORIO: Matriz 3 x 3	<table border="0"> <tr> <td>1</td><td>2</td><td>3</td></tr> <tr> <td>2</td><td>4</td><td>5</td></tr> <tr> <td>3</td><td>5</td><td>6</td></tr> </table>	1	2	3	2	4	5	3	5	6	Comparar (1,2) con (2,1): ambos = 2 Comparar (1,3) con (3,1): ambos = 3 Comparar (2,3) con (3,2): ambos = 5	Todos coinciden, es simétrica.
1	2	3										
2	4	5										
3	5	6										

16.- Ordenamiento rápido (QuickSort): Implementar QuickSort usando funciones recursivas.

```

Algoritmo QuickSort(A, inicio, fin)
  Si inicio < fin entonces
    pivote ← particion(A, inicio, fin)
    QuickSort(A, inicio, pivote-1)
    QuickSort(A, pivote+1, fin)
  FinSi
FinAlgoritmo

```

Función particion(A, inicio, fin)

```

x ← A[fin] // pivote
i ← inicio - 1
Para j desde inicio hasta fin-1 hacer
    Si A[j] <= x entonces
        i ← i + 1
        Intercambiar(A[i], A[j])
    FinSi
FinPara
Intercambiar(A[i+1], A[fin])
Retornar i+1
FinFunción

```

17.- Gestión de inventario con archivos: Crear archivo productos.txt , con los siguientes campos: Leer productos de un archivo, aplicar descuentos y generar un reporte final.

ALGORITMO GestionInventario

INICIO

```

ABRIR archivo "productos.txt"
LEER todos los productos (nombre, precio, stock) en una lista INVENTARIO

```

```

MOSTRAR "===== INVENTARIO DISPONIBLE ====="

```

```

PARA cada producto en INVENTARIO HACER

```

```

    MOSTRAR número, nombre, precio y stock

```

```

FIN PARA

```

```

CREAR lista CARRITO vacía

```

REPETIR

```

    MOSTRAR "Ingrese el número del producto que desea comprar"

```

```

    LEER ELECCION

```

```

    SI ELECCION no es válido ENTONCES

```

```

        MOSTRAR "Producto inválido"

```

```

        CONTINUAR

```

```

    FIN SI

```

```

    MOSTRAR "Ingrese la cantidad que desea comprar"

```

```

    LEER CANTIDAD

```

```

    SI CANTIDAD > stock disponible ENTONCES

```

```

        MOSTRAR "No hay suficiente stock"

```

```

        CONTINUAR

```

```

    FIN SI

```

```

    AGREGAR producto elegido y cantidad al CARRITO

```

```

    MOSTRAR "¿Desea comprar otro producto? (s/n)"

```

```

    LEER OPCION

```

```

    HASTA que OPCION sea 'n'

```

```

    MOSTRAR "Ingrese el nombre del cliente"

```

```

    LEER CLIENTE

```

```

    GENERAR CODIGO_COMPRA aleatorio (ejemplo: C12345)

```

CREAR archivo "reporte_CLIENTE_CODIGO_COMPRA.txt"

ESCRIBIR en archivo:

"===== REPORTE DE COMPRA ====="

"Cliente: CLIENTE"

"Código de compra: CODIGO_COMPRA"

TOTAL_COMPRA $\leftarrow$ 0

PARA cada producto en CARRITO HACER

SUBTOTAL $\leftarrow$ precio * cantidad

SI precio > 300 Y cantidad $\geq$ 2 ENTONCES

APLICAR descuento 2x1 (solo cobrar la mitad de las unidades)

SUBTOTAL $\leftarrow$ precio * (cantidad/2 + cantidad%2)

FIN SI

ESCRIBIR producto, precio, cantidad, subtotal en archivo

TOTAL_COMPRA $\leftarrow$ TOTAL_COMPRA + SUBTOTAL

FIN PARA

SI TOTAL_COMPRA > 220 ENTONCES

DESCUENTO $\leftarrow$ TOTAL_COMPRA * 0.10

TOTAL_COMPRA $\leftarrow$ TOTAL_COMPRA - DESCUENTO

ESCRIBIR "Descuento adicional del 10% aplicado: -DESCUENTO"

FIN SI

ESCRIBIR "TOTAL FINAL: TOTAL_COMPRA" en archivo

MOSTRAR "Compra realizada. Se generó el archivo: reporte_CLIENTE_CODIGO_COMPRA.txt"

FIN

18.- Matrices dinámicas con apuntadores: Crear y manipular matrices dinámicas usando doble puntero.

Algoritmo Matriz_Dinamica

Leer n, m

Crear matriz como doble puntero

Reservar memoria para n filas

Para cada fila i

Reservar memoria para m columnas

FinPara

Leer los elementos de la matriz

Mostrar la matriz

Liberar memoria de cada fila

Liberar memoria del arreglo de filas

FinAlgoritmo

19.- Ordenamiento combinado: Implementar Merge Sort y comparar su eficiencia con Bubble Sort.

Algoritmo MergeSort(A)

Si tamaño(A) > 1 entonces

mitad $\leftarrow$ tamaño(A)/2

izquierda $\leftarrow$ A[0..mitad-1]

derecha $\leftarrow$ A[mitad..fin]

```
MergeSort(izquierda)
MergeSort(derecha)
Combinar(izquierda, derecha, A)
FinSi
FinAlgoritmo
```

Comparación de eficiencia:

- **Bubble Sort:** $O(n^2)$. Muy lento en vectores grandes.
- **Merge Sort:** $O(n \log n)$. Mucho más rápido y eficiente.

En vectores pequeños (ej. 10 elementos) ambos funcionan bien.

En vectores grandes (ej. 10,000 elementos), Merge Sort es muchísimo más rápido.

21.- Dado un vector de números enteros, escriba una función que ordene los elementos del vector de tal forma que los números pares aparezcan antes que los números impares. Además, los números pares deberán estar ordenados de forma ascendente, mientras que los números impares deberán estar ordenados de forma descendente. Esto es, el vector {1,2,3,4,5,6} quedará como {2,4,6,5,3,1}.

Algoritmo OrdenarParImpar

```
Leer vector
Crear listaPares ← elementos pares del vector
Crear listaImpares ← elementos impares del vector
Ordenar listaPares en forma ascendente
Ordenar listaImpares en forma descendente
Concatenar listaPares + listaImpares → vectorFinal
Mostrar vectorFinal
```

FinAlgoritmo

PRUEBA DE ESCRITORIO:

Vector inicial = [10, 7, 4, 3, 8, 9, 2]

Pares: [10, 4, 8, 2] → orden ascendente → [2, 4, 8, 10]

Impares: [7, 3, 9] → orden descendente → [9, 7, 3]

Vector Final = [2, 4, 8, 10, 9, 7, 3]

22.- Escriba una función que tome un valor entero y devuelva el número con los dígitos invertidos. Por ejemplo, dado 7631, la función debe regresar 1367.

Función invertirNumero(n)

```
invertido ← 0
```

```
Mientras n ≠ 0 hacer
```

```
    digito ← n % 10
```

```
    invertido ← invertido * 10 + digito
```

```
    n ← n / 10
```

```
FinMientras
```

Retornar invertido

FinFunción

PRUEBA DE ESCRITORIO: 54027 $\rightarrow$ 72045

1. Último dígito = 7 $\rightarrow$ invertido = 7
2. Siguiente dígito = 2 $\rightarrow$ invertido = 72
3. Siguiente dígito = 0 $\rightarrow$ invertido = 720
4. Siguiente dígito = 4 $\rightarrow$ invertido = 7204
5. Siguiente dígito = 5 $\rightarrow$ invertido = 72045

23.- El máximo común divisor de dos enteros es el entero más grande que divide cada uno de los números. Escriba un programa mcd que devuelva el máximo común divisor de dos enteros.

Función MCD(a, b)

Mientras b $\neq$ 0 hacer

temp $\leftarrow$ b

b $\leftarrow$ a mod b

a $\leftarrow$ temp

FinMientras

Retornar a

FinFunción

PRUEBA DE ESCRITORIO: a = 84, b = 30

84 mod 30 = 24 $\rightarrow$ (a=30, b=24)

30 mod 24 = 6 $\rightarrow$ (a=24, b=6)

24 mod 6 = 0 $\rightarrow$ (a=6, b=0)

MCD(84,30) = 6

24.- Escriba una función puntoCalidad que tome el promedio de un estudiante y devuelva 4 si el promedio del estudiante está entre 90-100; 3 si el promedio es 80-89; 2 si el promedio es 70-79; 1 si el promedio es 60-69, y 0 si el promedio es menor que 60.

Algoritmo PuntoCalidadNotas

Leer cantidad de notas

suma $\leftarrow$ 0

Para i desde 1 hasta cantidad hacer

Leer nota

suma $\leftarrow$ suma + nota

FinPara

promedio $\leftarrow$ suma / cantidad

Si promedio ≥ 90 y promedio ≤ 100 entonces

calidad $\leftarrow 4$

Sino si promedio ≥ 80 entonces

calidad $\leftarrow 3$

Sino si promedio ≥ 70 entonces

calidad $\leftarrow 2$

Sino si promedio ≥ 60 entonces

calidad $\leftarrow 1$

Sino

calidad $\leftarrow 0$

Mostrar promedio y calidad

FinAlgoritmo